AWS Cost Optimization Dashboard

Internship Project Submitted to

Codec Technologies India

Prepared by

Aditya Banerjee
B.Tech. Computer Science and Engineering
University of Kalyani

Internship Domain

Cloud Computing

Submitted as part of Internship Program Codec Technologies India 2025–2026

Declaration

I hereby declare that the project entitled **"AWS Cost Optimization Dashboard"** is an original work carried out by me during my internship at Codec Technologies India. This project focuses on building a cloud-native FinOps solution to analyze AWS infrastructure,detect idle resources, recommend rightsizing, and estimate cost savings.

Acknowledgment

I express my sincere gratitude to Codec Technologies India. for providing me the opportunity to work on a real-world cloud cost optimization project. This internship enabled me to apply concepts of Cloud Computing, DevOps, and FinOps to design a production-style solution.

Abstract

Cloud adoption has increased rapidly, but inefficient resource utilization often leads to unnecessary operational costs. This project presents an **AWS Cost Optimization Dashboard**, a cloud-native platform designed to analyze infrastructure usage across multiple AWS regions,detect idle resources, and recommend optimization strategies.The system collects utilization metrics from Amazon CloudWatch, cost data from AWS Cost Explorer,and applies an analytics engine to identify waste and estimate potential savings.It is implemented using FastAPI microservices deployed via Docker containers,while Grafana provides real-time visualization dashboards.Automation is achieved through AWS Lambda scheduling, reports are stored in Amazon S3,and alerts are delivered via Amazon SNS.

Contents

1 Introduction **5**

2 Literature Review **6**

3 Proposed Work **7**

3.1 System Overview 7

3.2 Architecture 7

4 Technology Stack **10**

5 Results and Discussion **11**

5.1 Idle Resource Detection Dashboard 11

5.2 Rightsizing Recommendation Engine 11

5.3 Cost Savings Estimation 13

5.4 Full Optimization Report API Output 13

6 Conclusion **16**

7 Future Scope **17**

Chapter 1

Introduction

Organizations today rely on cloud infrastructure for scalability and agility.However, lack of visibility into resource utilization leads to over-provisioning,idle resources, and rising operational expenses.FinOps has emerged to bridge engineering and finance by enabling data-driven cost optimization.This project builds an automated AWS cost analysis platform to continuously monitor usage, detect inefficiencies, and provide actionable recommendations.

Chapter 2

Literature Review

Traditional cost monitoring relies on manual reporting and static dashboards,which lack automation and real-time insight. Modern FinOps solutions integrate monitoring, analytics, and automation to enable proactive optimization.Cloud providers offer tools such as CloudWatch and Cost Explorer,but combining these into a unified optimization workflow remains a challenge. This project addresses that gap by integrating monitoring, analytics,and automation into a single system.

Chapter 3

Proposed Work

3.1 System Overview

The system is designed as a containerized cloud analytics platform that enables automated monitoring, analysis, and optimization of AWS infrastructure costs.It integrates multiple AWS services with a microservice-based backend to deliver actionable FinOps insights.

The major components of the system include:

- **FastAPI Backend** – Collects AWS metrics and exposes optimization APIs.
- **Optimization Engine** – Detects idle resources and provides rightsizing recommendations.
- **Dockerized Deployment** – Ensures portability and reproducibility of the platform.
- **Grafana Dashboards** – Visualizes optimization insights in real time.
- **AWS Lambda** – Triggers scheduled analysis automatically.
- **Amazon S3** – Stores generated optimization reports.
- **Amazon SNS** – Sends alert notifications and recommendations.

3.2 Architecture

The overall architecture of the AWS Cost Optimization Dashboard is illustrated in Figure 3.1. The system integrates monitoring, analytics, automation, and visualization layers into a unified workflow.

Workflow Description

The architecture operates through the following sequence:

1. AWS utilization metrics are collected from **Amazon CloudWatch** and cost data from **AWS Cost Explorer**.
2. The FastAPI-based analytics service processes this data using the optimization engine to detect idle or underutilized EC2 instances.
3. Optimization recommendations and savings estimations are generated.
4. Reports are stored centrally in **Amazon S3** for persistence and auditing.
5. Alert notifications are delivered through **Amazon SNS** to inform stakeholders.
6. **AWS Lambda** triggers this workflow periodically for continuous monitoring.
7. **Grafana** consumes the API outputs to present real-time dashboards for visualization and decision-making.

This architecture ensures scalability, automation, and clear separation between data collection, processing, and visualization layers, making the platform suitable for production-scale FinOps environments.

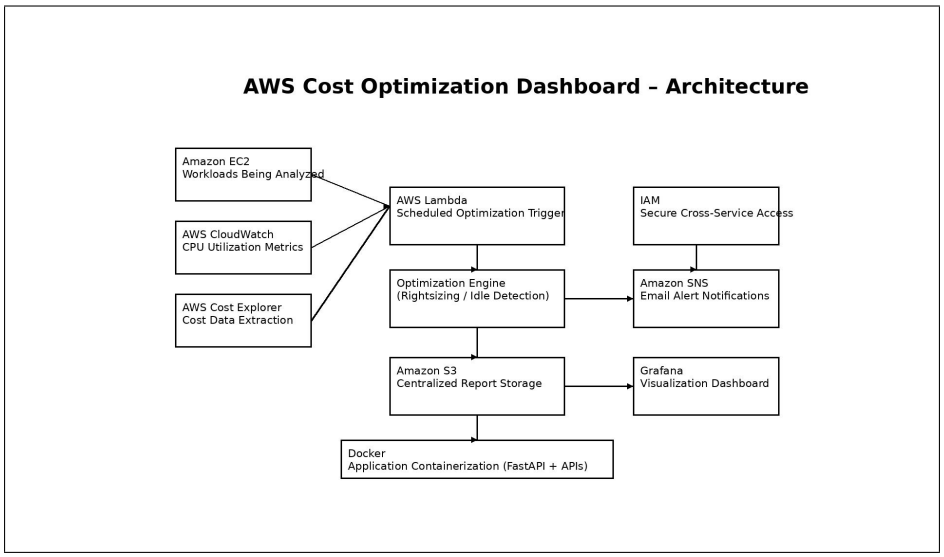

Figure 3.1: System Architecture of AWS Cost Optimization Dashboard

Chapter 4

Technology Stack

- AWS EC2 – Workloads analyzed
- AWS CloudWatch – Metrics collection
- AWS Cost Explorer – Cost analytics
- AWS Lambda – Scheduled execution
- Amazon S3 – Report storage
- Amazon SNS – Email alerts
- FastAPI – Backend APIs
- Docker Docker Compose – Deployment
- Grafana – Visualization
- Python (Boto3, Pandas) – Data processing

Chapter 5

Results and Discussion

This chapter presents the experimental results obtained from the implementation of the**AWS Cost Optimization Dashboard**. The system was tested across multiple AWS regions including Mumbai(ap-south-1),Frankfurt(eu-central-1),Sao Paulo(sa-east-1) to analyze EC2 utilization, detect idle resources, recommend rightsizing, and estimate monthly cost savings.The backend services were deployed using Docker containers, and the dashboards were visualized using Grafana. The automation workflow was triggered through AWS Lambda,while reports were stored in Amazon S3 and alerts delivered via Amazon SNS.

5.1 Idle Resource Detection Dashboard

The above dashboard visualizes EC2 instances identified as idle across the analyzed AWS regions. Instances that are in a stopped state or exhibit negligible CPU utilization are flagged by the optimization engine as cost-inefficient resources.Using CloudWatch utilization metrics and Cost Explorer pricing data, the system recommends terminating such instances to prevent unnecessary storage and allocation charges. This helps organizations eliminate hidden cloud waste and improve financial governance under FinOps practices.

5.2 Rightsizing Recommendation Engine

The optimization engine analyzes CPU utilization metrics from CloudWatch to determine whether instances are over-provisioned. It then recommends downsizing to reduce unnecessary compute costs.

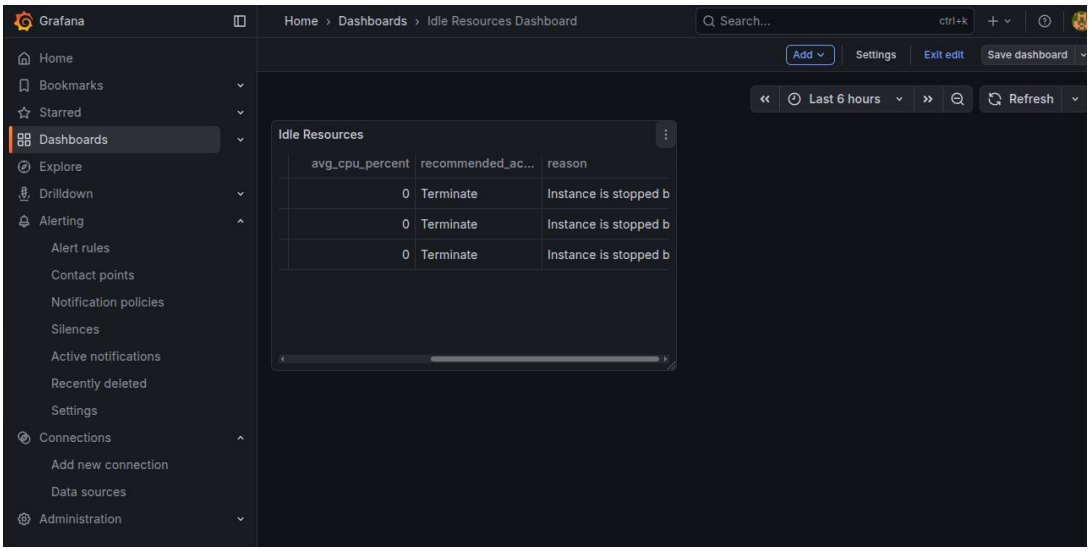

Figure 5.1: Idle Resource Detection Dashboard

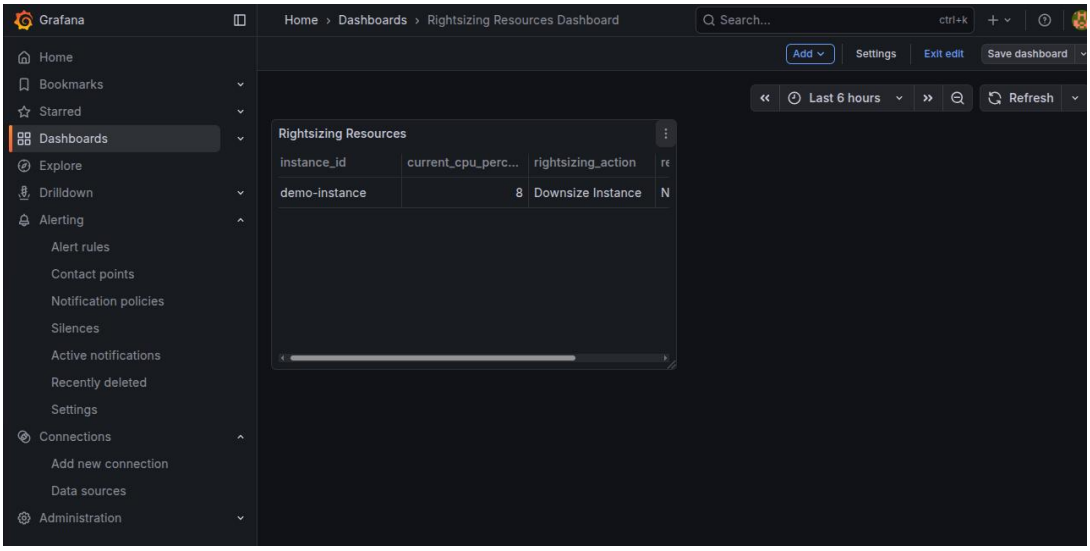

Figure 5.2: Idle Resource Detection Dashboard

5.3 Cost Savings Estimation

The system calculates projected monthly savings by correlating instance pricing data with detected inefficiencies using AWS Cost Explorer datasets.

5.4 Full Optimization Report API Output

The FastAPI service aggregates insights into a single JSON response consumed by Grafana and stored in S3 for historical tracking.

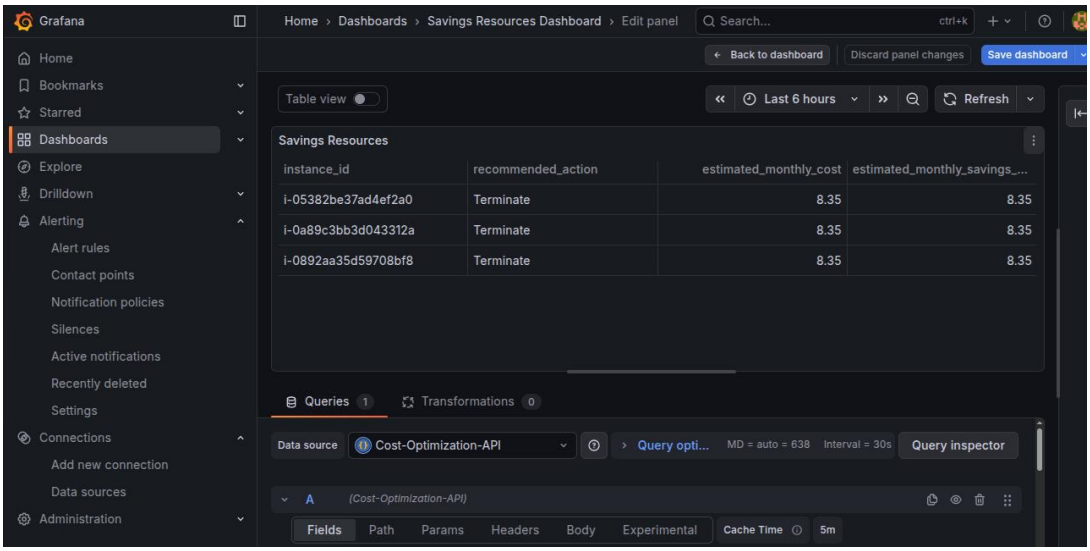

Figure 5.3: Idle Resource Detection Dashboard

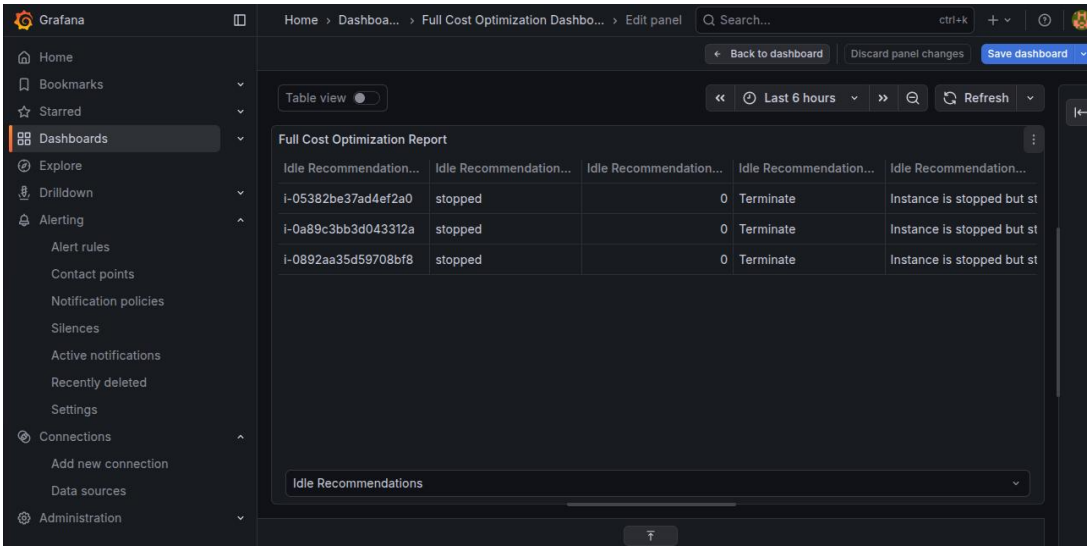

Figure 5.4: Idle Resource Detection Dashboard

The experimental results confirm that the platform can continuously monitor cloud infrastructure, detect inefficiencies, and provide actionable FinOps insights. This validates the effectiveness of combining monitoring, analytics, and automation into a single cost-optimization workflow.

Chapter 6

Conclusion

The AWS Cost Optimization Dashboard demonstrates how FinOps automation can transform cloud cost management into a proactive optimization strategy.By integrating monitoring, analytics, and automation, the platform improves resource efficiency and financial governance.

Chapter 7

Future Scope

- Kubernetes cost analysis integration
- AI-based predictive scaling
- Automated remediation workflows
- Multi-cloud optimization support
- Cost anomaly detection using machine learning

Bibliography

[1] Amazon Web Services Documentation, *AWS Cost Management Guide*

[2] FinOps Foundation, *Cloud Financial Management Principles*

[3] Grafana Labs Documentation

[4] FastAPI Framework Documentation